



THAPAR INSTITUTE
OF ENGINEERING & TECHNOLOGY
(Deemed to be University)

MID WAY REPORT

on

DevSecOps Code Governance Portal

FACULTY MENTOR

Dr. Sachin Kansal

INDUSTRIAL MENTOR

Mr. Bharat Panwar

Bharat Heavy Electricals Limited (BHEL)

SUBMITTED BY

Prakriti Kain

Roll No. 102203237

Computer Science and Engineering Department
Thapar Institute of Engineering & Technology, Patiala

Table of Contents

Report Overview	1
1. Introduction / Background	2
2. Problem Statement / Domain	2
3. Objectives of the Work	2
4. Techniques / Tools / Technologies Used	3
5. Block Diagram / Architecture / Methodology	4
Future Scope	5

Report Overview

This mid-way report presents the progress of the work carried out on the project titled "DevSecOps Code Governance Portal." The report documents the background of the problem being addressed, the system architecture, the techniques and technologies used, and the contribution made in building a web portal that sits on top of a CI/CD pipeline, automatically scanning every commit, consolidating the security findings, and enforcing a role-based approval gate between the Security and Operations teams — all through a single push → scan → review → approve → deploy loop.

Every code push is automatically built and scanned (SAST, SCA, DAST) with no manual step.

A server-enforced approval gate ensures only Security-approved commits can be deployed.

Findings from SonarQube, OWASP Dependency-Check, and OWASP ZAP are unified into one dashboard.

1. Introduction / Background

Modern software teams release code rapidly, but security review often lags behind delivery. In many organizations there is no enforced checkpoint between development, security review, and deployment, which means vulnerable or unreviewed code can reach production. Security tools exist, but their findings are scattered across separate dashboards and are rarely acted upon in a controlled, auditable way.

The DevSecOps Code Governance Portal addresses this gap with a web platform that integrates a Git server, an automation server, and three classes of security scanners into one automated pipeline. It guides every commit from being pushed, to being built and scanned, to being reviewed by Security, and finally to being deployed by Operations — closing the loop between detecting a vulnerability and controlling whether the code ships.

2. Problem Statement / Domain

The project sits at the intersection of DevSecOps, application security, and CI/CD automation. Despite the wide availability of scanning tools, individual teams lack a single channel that connects assessment, review, approval, and deployment. Findings are fragmented, the link between a commit and its vulnerabilities is unclear, and deployment decisions are made informally without an auditable record.

Key challenges identified

- Integrating heterogeneous scanners — SAST, SCA, and DAST — into one automated pipeline.
- Normalizing findings from different tools into a single, comparable severity model.
- Enforcing a deployment gate on the server side so unapproved code can never be shipped.
- Making the approval and deployment lifecycle traceable and auditable for every commit.

3. Objectives of the Work

- To build a web portal that records every commit together with its security scan results.
- To automate SAST, SCA, and DAST scanning on every push through a CI/CD pipeline.
- To enforce a role-based approval gate so only Security-approved commits reach Operations.
- To consolidate multi-tool findings into one severity-classified dashboard.
- To provide secure, role-segregated access for Developer, Security, and Operations users.
- To capture an auditable trail of approvals, rejections, and deployments per commit.

4. Techniques / Tools / Technologies Used

4.1 Portal Application

The portal is built with Java Servlets, JSP, and JDBC on Apache Tomcat 10 (Jakarta namespace). Authentication is session-based with BCrypt password hashing, and a server-side servlet Filter maps URL prefixes to roles. A MySQL database is the single source of truth for users, commits, scan results, approvals, and deployments.

4.2 CI/CD Pipeline

A Gitea Git server fires a webhook on every push to a Jenkins automation server, which checks out the commit, builds it with Maven, and orchestrates the scan stages — keeping the whole build-and-scan process automatic and reproducible.

4.3 Security Scanning

Static analysis (SAST) is performed by SonarQube, software-composition analysis (SCA) by OWASP Dependency-Check against the NVD database, and dynamic analysis (DAST) by OWASP ZAP run against a staging deployment of the application.

4.4 Integration & Data

Jenkins POSTs a results summary to the portal's REST endpoint. The portal pulls SonarQube findings through its Web API, parses the Dependency-Check and ZAP reports, and writes commits, scan results, and a pending approval to MySQL. The receiver is idempotent on the commit hash so repeated runs do not create duplicates.

4.5 Infrastructure & Delivery

The entire stack — MySQL, Gitea, Jenkins, SonarQube, the portal, and a staging Tomcat — is containerized with Docker Compose for reproducible setup, and the application logic is covered by JUnit tests.

Key analytical focus areas

- Automated multi-tool security scanning on every commit
- Severity normalization across SAST, SCA, and DAST
- Role-based access control and the deployment approval gate
- Commit-level traceability and audit

Tools used

Java

Servlets / JSP

MySQL

Jenkins

Gitea

SonarQube

Dependency-Check

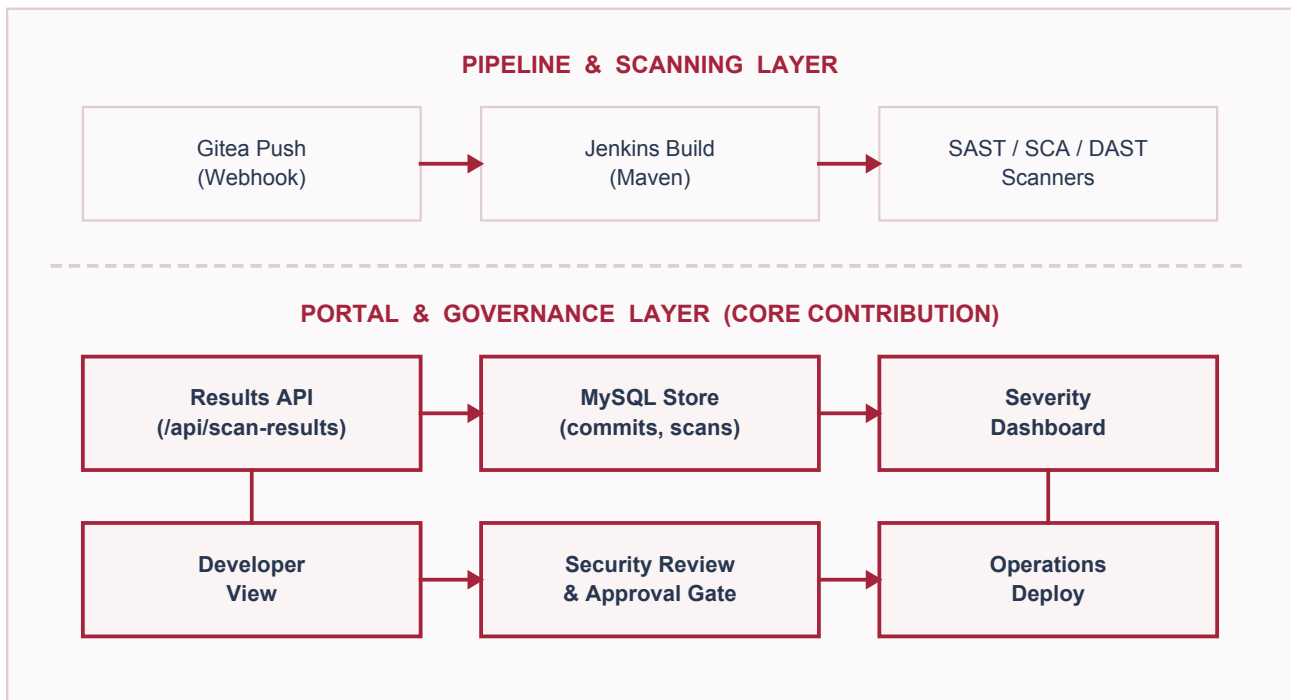
OWASP ZAP

Docker

Maven

5. Block Diagram / Architecture / Methodology

5.1 System Overview



5.2 Methodology Flow

- A developer pushes code to the Gitea repository.
- Gitea's webhook triggers the Jenkins pipeline against that exact commit.
- Jenkins builds the project and runs SAST (SonarQube), SCA (Dependency-Check), and DAST (ZAP).
- The pipeline POSTs a results summary to the portal's REST endpoint.
- The portal stores the commit, its scan results, and a PENDING approval in MySQL.
- The Security team reviews the findings and approves or rejects the commit, with a comment.
- Only approved commits appear in the Operations deployment queue.
- Operations deploys the approved code and the status is recorded; rejected commits never reach Operations.

The methodology ensures deployment is governed by an explicit security review — not a manual, error-prone handoff — with every commit's findings, decision, and deployment status fully traceable.

Future Scope

- Add Single Sign-On (SSO) and fine-grained, per-repository permissions.
- Introduce policy-as-code quality gates that automatically block on any critical finding.
- Add trend dashboards and metrics such as mean-time-to-remediate and findings over time.
- Integrate additional scanners — secret scanning, container-image scanning, and IaC scanning.
- Support multi-repository and multi-team governance at organization scale.
- Add email / Slack notifications for pending approvals and deployment events.